

SMART JUNCTION SIMULATION

Adaptive Traffic Control System

C++ Object-Oriented Programming
Mini Project Report

1. Abstract

This project implements an Adaptive Traffic Control System in C++ using Object-Oriented Programming. It simulates a four-way junction with density-based signal scheduling, emergency vehicle preemption, automated overspeed fine detection, and persistent file-based fine recording - demonstrating over 20 distinct C++ OOP and STL concepts in a single cohesive application.

The system is menu-driven with six options, accepting vehicles both manually and from a formatted input file. Fine records are written to fines.txt during simulation and can be reviewed independently. The project targets practical demonstration of abstraction, inheritance, runtime polymorphism, templates, exception handling, STL containers, file I/O, chrono timing, and heap memory management.

2. Problem Statement

Conventional fixed-cycle traffic signals create three fundamental problems in urban junction management:

- **Artificial Congestion:** Fixed-time signals allocate equal green time regardless of lane traffic volume. Lanes with low density get the same time as congested lanes, creating artificial queuing delays.
- **Emergency Delays:** Traditional systems cannot detect or prioritize emergency vehicles, causing critical delays for ambulances and fire trucks passing through intersections.
- **Manual Enforcement:** Speed enforcement depends entirely on manual observation, making it resource-intensive, inconsistent, and easily circumvented without physical infrastructure.

3. Objectives

- Implement a four-road smart junction simulation using C++ OOP with a clear, extensible class hierarchy.
- Demonstrate density-based green light scheduling that prioritizes high-traffic roads dynamically at runtime.
- Implement emergency vehicle preemption using `dynamic_cast` and a strongly-typed `Priority` enum class.
- Automate overspeed detection per vehicle type and log fine records persistently to fines.txt using file I/O.
- Support both manual vehicle entry and batch file-based vehicle loading with full exception handling.
- Demonstrate all required OOP concepts: inheritance, polymorphism, templates, STL, file I/O, exceptions, namespaces, chrono timing, and memory management.

4. System Design

4.1 Class Hierarchy

The system centres on an abstract Vehicle base class from which four concrete vehicle types are derived. Supporting classes manage road state, signal control, and junction aggregation.

Class / Entity	Type	Role
Vehicle	Abstract Base	Holds type, speed, number. Declares pure virtual move() and virtual getPriority().
Car	Derived	Concrete vehicle - speed limit 80 km/h; inherits all Vehicle attributes.
Bike	Derived	Concrete vehicle - speed limit 60 km/h; inherits all Vehicle attributes.
Truck	Derived	Concrete vehicle - speed limit 50 km/h; inherits all Vehicle attributes.
Ambulance	Derived	Emergency vehicle - overrides getPriority() to EMERGENCY; exempt from fines.
TrafficLight	Utility	Encapsulates signal color state (RED/YELLOW/GREEN) with get/set methods.
Road	Container	Holds vehicle queue + signal state + road name. Exposes displayStatus().
Junction	Aggregation	Contains vector of 4 Roads: North, South, East, West.
TrafficController	Controller	Core logic: emergency check, density sort, signal cycle, fine recording.

4.2 System Architecture

A Junction object aggregates four Road objects (North, South, East, West). Each Road maintains a queue<Vehicle*> for FIFO ordering. At runtime, a TrafficController iterates over roads, checks for emergency vehicles via dynamic_cast, sorts remaining roads by queue density using std::sort with a lambda, then drives each road through the GREEN -> YELLOW -> RED signal cycle. After each vehicle clears, checkOverspeed() records fines to fines.txt.

4.3 Adaptive Signal Logic

The controller's manage() function applies a two-tier adaptive scheduling strategy:

- Tier 1 - Emergency Priority: Roads containing an Ambulance are detected via dynamic_cast and placed at the front of the processing queue unconditionally, before any other road is served.
- Tier 2 - Density Sort: All remaining roads are sorted in descending order of queue size using std::sort with a lambda comparator. The busiest roads receive the signal earliest.
- Signal Cycle: GREEN (10-second countdown) -> YELLOW (5-second countdown) -> RED (instant). Countdown uses chrono::milliseconds(300) sleep per tick for realistic console output.

4.4 Menu Structure

The program runs in a continuous while(true) loop presenting six menu choices. Each maps to a distinct subsystem:

Choice	Feature	Description
1	Add Vehicle Manually	User enters road, vehicle type, number plate, and speed via console prompts.
2	Load Vehicles From File	Reads a pre-formatted vehicles.txt file; clears existing vehicles first.
3	Run Simulation	Executes adaptive signal control: ambulance preemption + density sort + fine logging.
4	Display Junction Status	Shows current signal state (RED/GREEN/YELLOW) of all four roads.
5	View Fines	Reads and displays all recorded fines from fines.txt.
6	Exit	Deletes fines.txt, frees all heap memory, and terminates the program.

5. OOP & C++ Concepts Implementation

The table below maps all 20 OOP and C++ language concepts demonstrated in the project to their specific code implementation.

OOP Concept	C++ Feature Used	Implementation in Project
Abstraction	Abstract class / pure virtual	Vehicle is abstract; move() = 0 forces all derived classes to implement movement.
Inheritance	public inheritance	Car, Bike, Truck, Ambulance all publicly inherit from Vehicle, reusing type/speed/number.
Polymorphism	virtual / override	move() and getPriority() are virtual. Ambulance overrides getPriority() -> EMERGENCY.
Encapsulation	private members + getters	Road queue and TrafficLight state are private; accessed only via defined methods.
Constructors	Default / Parameterized / Copy	Vehicle has all three constructors demonstrating constructor overloading.
Friend Function	friend keyword	showVehicle() accesses private Vehicle members directly without being a class member.
Operator Overload	operator> (commented)	Overloaded > on Vehicle for speed-based comparison; commented for optional activation.
Templates	template<typename T>	getMax<T>(a,b) provides generic type-safe max - used in overspeed speed comparison.

OOP Concept	C++ Feature Used	Implementation in Project
Namespace	namespace TrafficUtils	calculateFine() is scoped inside TrafficUtils namespace, avoiding global pollution.
Enum Class	enum class Priority	Strongly-typed NORMAL / EMERGENCY enum prevents implicit integer conversion.
Exception Handling	try / catch / throw	loadVehiclesFromFile() throws runtime_error; main() catches and prints the message.
Dynamic Cast	dynamic_cast<Ambulance*>	hasAmbulance() safely downcasts Vehicle* to Ambulance* to detect emergency vehicles.
Static Cast	static_cast<int>	Fine value cast to int explicitly after calculateFine() to ensure correct type.
STL - queue	queue<Vehicle*>	Each Road holds a FIFO queue of vehicle pointers - first in, first to clear junction.
STL - vector	vector<Road>, vector<Road*>	Junction stores Road objects; controller builds a Road* processing order vector.
STL - sort	sort() + lambda	rem vector sorted descending by queue size for density-based scheduling.
File I/O (write)	ofstream, ios::app	checkOverspeed() appends fine records to fines.txt after each simulation run.
File I/O (read)	ifstream, getline()	loadVehiclesFromFile() reads vehicle data from a CSV-style input file.
Chrono / Thread	this_thread::sleep_for	countdown() simulates real-time signal durations using millisecond sleep intervals.
Forward Declarations	class X; declarations	All major classes declared upfront to resolve circular dependency in headers.

6. Code Walkthrough

6.1 Forward Declarations & Global Utilities

All major classes are forward-declared at the top to resolve circular references. A template function, namespace, and enum class are also defined globally before any class definition.

```
Forward Declarations, Template, Namespace, Enum

// Forward declarations - resolve circular class references
class Vehicle; class Road; class Junction;
class Car; class Bike; class Truck; class Ambulance;

// Template function - generic max comparison
template <typename T>
T getMax(T a, T b) { return (a > b) ? a : b; }

// Namespace - isolate utility logic
namespace TrafficUtils {
    int calculateFine(int overSpeed) { return overSpeed * 100; }
```

```
}

// Enum class - strongly typed signal priority
enum class Priority { NORMAL, EMERGENCY };
```

6.2 Abstract Base Class — Vehicle

Vehicle is the abstract foundation of the hierarchy. It has three constructors (default, parameterized, copy), protected members with public getters, a pure virtual move() function, and a friend function declaration. The virtual destructor ensures correct cleanup of derived objects through base pointers.

```
Abstract Base Class - Vehicle

class Vehicle
{
protected:
    string type;
    int speed;
    string number;

public:
    Vehicle() : type("Unknown"), speed(0), number("NA") {}           // default constructor
    Vehicle(string t, int s, string n) : type(t), speed(s), number(n) {} // parameterized
    Vehicle(const Vehicle& v)                                           // copy constructor
    { type = v.type; speed = v.speed; number = v.number; }

    virtual void move() = 0;                                           // pure virtual - enforces polymorphism
    virtual Priority getPriority() { return Priority::NORMAL; }

    string getType() { return type; }
    int getSpeed() { return speed; }
    string getNumber() { return number; }

    virtual ~Vehicle() {}                                           // virtual destructor
    friend void showVehicle(Vehicle& v); // friend declaration
};

// Friend function - accesses private members directly
void showVehicle(Vehicle& v)
{ cout << v.number << " (" << v.type << ")"; }
```

6.3 Derived Vehicle Classes

Four concrete classes inherit from Vehicle. Each overrides move(). Ambulance additionally overrides getPriority() to return Priority::EMERGENCY - the key hook used by dynamic_cast in the controller to detect emergency vehicles.

```
Derived Classes - Car, Bike, Truck, Ambulance

class Car : public Vehicle {
public:
    Car(int s, string n) : Vehicle("Car", s, n) {}
    void move() override {}
};

class Bike : public Vehicle {
public:
    Bike(int s, string n) : Vehicle("Bike", s, n) {}
```

```
        void move() override {}
    };
    class Truck : public Vehicle {
    public:
        Truck(int s, string n) : Vehicle("Truck", s, n) {}
        void move() override {}
    };
    class Ambulance : public Vehicle {
    public:
        Ambulance(string n) : Vehicle("Ambulance", 100, n) {}
        void move() override {}
        Priority getPriority() override { return Priority::EMERGENCY; } // overridden
    };
```

6.4 Road & Junction Classes

Road encapsulates a `queue<Vehicle*>` for FIFO processing and manages its signal state string. Junction aggregates four Road objects representing the four directions of the simulated intersection.

```
Road & Junction Classes

class Road {
    string name;
    queue<Vehicle*> vehicles;    // STL FIFO queue
    string signalState;
public:
    Road(string n) : name(n), signalState("RED") {}
    void addVehicle(Vehicle* v) { vehicles.push(v); }
    queue<Vehicle*>& getQueue() { return vehicles; }
    string getName() { return name; }
    void setSignal(string s) { signalState = s; }
    void displayStatus() { cout << name << "->" << signalState << endl; }
};

class Junction {
    vector<Road> roads;
public:
    Junction() {
        roads.push_back(Road("North")); roads.push_back(Road("South"));
        roads.push_back(Road("East")); roads.push_back(Road("West"));
    }
    vector<Road>& getRoads() { return roads; }
};
```

6.5 Overspeed Detection — Template, static_cast, Namespace, File I/O

`checkOverspeed()` combines four distinct C++ features: the generic `getMax<T>()` template for speed comparison, `static_cast<int>` for explicit type conversion, the `TrafficUtils` namespace for fine calculation, and `ofstream` in append mode for persistent fine recording.

```
checkOverspeed() - Template + static_cast + Namespace + File I/O

void checkOverspeed(Vehicle* v)
{
    ofstream file("fines.txt", ios::app);    // append mode - preserves previous records
    int limit = 0;

    if (v->getType() == "Car")    limit = 80;
```

```
else if (v->getType() == "Bike") limit = 60;
else if (v->getType() == "Truck") limit = 50;
else return; // Ambulance - exempt from fines

int maxVal = getMax(v->getSpeed(), limit); // template function used here
if (maxVal == v->getSpeed() && v->getSpeed() > limit)
{
    int fine = static_cast<int>( // static_cast for explicit conversion
        TrafficUtils::calculateFine(v->getSpeed() - limit)); // namespace usage

    file << "Vehicle: " << v->getNumber()
        << " | Type: " << v->getType()
        << " | Fine: " << fine << endl;
}
file.close();
}
```

6.6 TrafficController — dynamic_cast, STL sort, Lambda, Countdown

The controller is the core of the simulation. It uses `dynamic_cast` for safe runtime type checking, builds a priority-ordered `Road*` vector, sorts by density using a lambda, drives the GREEN/YELLOW/RED signal cycle, and manages heap memory via `delete`.

```
TrafficController - dynamic_cast + STL sort + Lambda + Memory Management

class TrafficController {
    TrafficLight* light;

    bool hasAmbulance(queue<Vehicle*> q) {
        while (!q.empty()) {
            Ambulance* amb = dynamic_cast<Ambulance*>(q.front()); // runtime type check
            if (amb) return true;
            q.pop();
        }
        return false;
    }

    void runSignal(Road& road, string state, int sec, bool timer) {
        light->setState(state); road.setSignal(state);
        cout << "
" << road.getName() << "->" << state;
        if (timer) { cout << " "; countdown(sec); }
        else cout << endl;
    }

public:
    TrafficController() { light = new TrafficLight(); }
    ~TrafficController() { delete light; }

    void manage(Junction& j) {
        // Tier 1: Emergency roads first
        vector<Road*> order;
        for (auto& r : j.getRoads())
            if (hasAmbulance(r.getQueue())) order.push_back(&r);

        // Tier 2: Density-based sort for remaining roads
        vector<Road*> rem;
        for (auto& r : j.getRoads()) rem.push_back(&r);
        sort(rem.begin(), rem.end(),
```

```
        [(Road* a, Road* b) {
            return a->getQueue().size() > b->getQueue().size(); // lambda
        });
    order.insert(order.end(), rem.begin(), rem.end());

    // Signal cycle for each road
    for (auto road : order) {
        auto& q = road->getQueue();
        if (q.empty()) continue;
        runSignal(*road, "GREEN", 10, true);
        runSignal(*road, "YELLOW", 5, true);
        runSignal(*road, "RED", 0, false);
        while (!q.empty()) {
            q.front()->move();
            checkOverspeed(q.front());
            delete q.front(); // heap memory freed
            q.pop();
        }
    }
};
```

6.7 File Loading — Exception Handling

`loadVehiclesFromFile()` throws a `runtime_error` if the file cannot be opened. The try-catch block in `main()` catches it gracefully, prints the error, and returns to the menu without crashing.

```
File Loading + Exception Handling

void loadVehiclesFromFile(Junction& junction, string filename)
{
    ifstream file(filename);
    if (!file) throw runtime_error("Error: Cannot open file!"); // throw

    string header; getline(file, header); // skip header row

    int r, t, s; string num;
    while (file >> r >> t >> s >> num) {
        Road& road = junction.getRoads()[r - 1];
        if (t == 1) road.addVehicle(new Car(s, num));
        else if (t == 2) road.addVehicle(new Bike(s, num));
        else if (t == 3) road.addVehicle(new Truck(s, num));
        else if (t == 4) road.addVehicle(new Ambulance(num));
    }
    cout << "Loaded from file
";
}

// In main() - exception caught:
try {
    clearJunction(junction);
    loadVehiclesFromFile(junction, filename);
}
catch (exception& e) { cout << e.what() << endl; } // catch
```

6.8 Countdown Timer — Chrono & Thread

The countdown simulates real signal duration. `this_thread::sleep_for()` with `chrono::milliseconds(300)` pauses execution for 300ms per tick, producing visible per-second countdown output on the console.

```
countdown() - Chrono & Thread

void countdown(int seconds)
{
    while (seconds > 0)
    {
        cout << seconds << " ";
        this_thread::sleep_for(chrono::milliseconds(300)); // chrono + thread
        seconds--;
    }
    cout << endl;
}
```

7. Speed Limits & Fine Structure

The system enforces per-vehicle-type speed limits. Fines are calculated as a linear function of excess speed using the `TrafficUtils::calculateFine()` function.

Vehicle Type	Speed Limit (km/h)	Fine Formula	Example
Car	80	Fine = (speed - 80) x 100	95 km/h -> Rs. 1500
Bike	60	Fine = (speed - 60) x 100	80 km/h -> Rs. 2000
Truck	50	Fine = (speed - 50) x 100	80 km/h -> Rs. 3000
Ambulance	Exempt	No fine	Emergency vehicle

Fine Formula: Fine (Rs.) = (Actual Speed - Speed Limit) x 100

8. Input File Format — vehicles.txt

When the user selects Option 2 (Load Vehicles From File), the program reads a space-separated text file. The first row is a header line and is consumed by `getline()` — it is never parsed as data.

8.1 Column Specification

Column	Field	Accepted Values	Notes
Column 1	Road	1 = North 2 = South 3 = East 4 = West	Integer 1-4 only
Column 2	Type	1 = Car 2 = Bike 3 = Truck 4 = Ambulance	Integer 1-4 only
Column 3	Speed	Any positive integer (km/h)	Set 0 for Ambulance - constructor ignores this value

Column	Field	Accepted Values	Notes
Column 4	Number	Vehicle registration number string	No spaces; use hyphens for compound names

8.2 Sample Input File

The sample below loads 8 vehicles across all four roads, including one Ambulance on East road (Road 3). This file is used to generate the sample output shown in Section 10.

```
vehicles.txt - Sample Input File

Road  Type  Speed  Number
1      1      95     KA01AB1234
1      2      55     KA01CD5678
2      1      70     TN05EF9012
2      3      80     TN05GH3456
3      4       0     AMBU-001
3      1      60     MH12IJ7890
4      2      80     DL08KL2345
4      3      65     GJ09MN6789
```

8.3 Input File Rules

- The header row (Road Type Speed Number) is always required - the program reads and discards it with `getline()`.
- Columns are whitespace-separated. The `>>` operator skips leading whitespace automatically.
- Ambulance rows must have Speed = 0 or any dummy value - the Ambulance constructor hardcodes speed as 100 internally.
- Vehicle number must be a single token with no spaces. Use hyphens for compound names, e.g. AMBU-001 or FIRE-02.
- If the filename is wrong or the file is not found, a `runtime_error` is thrown, caught in `main()`, printed to console, and the program continues safely.
- When Option 2 is selected, `clearJunction()` is called first - all existing vehicles are deleted from memory before the file is loaded.

9. Output File Format — fines.txt

The `fines.txt` file is automatically created (or cleared) at the start of every simulation run (Option 3). Each vehicle that exceeds its speed limit appends one line to this file via `ofstream` in `ios::app` mode.

9.1 Fine Record Format

Each fine record follows a fixed pipe-delimited single-line format:

```
Record Format
```

```
Vehicle: <number> | Type: <type> | Fine: <amount>
```

Field	Data Source	Example Value
Vehicle: <number>	v->getNumber() - registration plate string	Vehicle: KA01AB1234
Type: <type>	v->getType() - class-level type string	Type: Car
Fine: <amount>	TrafficUtils::calculateFine() return value (int)	Fine: 1500

9.2 Sample fines.txt Output

The following is the exact content written to fines.txt after running a simulation with the vehicles.txt sample from Section 8.2:

```
fines.txt - Output File

Vehicle: KA01AB1234 | Type: Car | Fine: 1500
Vehicle: TN05GH3456 | Type: Truck | Fine: 3000
Vehicle: DL08KL2345 | Type: Bike | Fine: 2000
```

9.3 Fine Calculation Breakdown

- KA01AB1234 Car 95 km/h: $(95 - 80) \times 100 = \text{Rs. } 1500$ [over limit by 15]
- TN05GH3456 Truck 80 km/h: $(80 - 50) \times 100 = \text{Rs. } 3000$ [over limit by 30]
- DL08KL2345 Bike 80 km/h: $(80 - 60) \times 100 = \text{Rs. } 2000$ [over limit by 20]
- AMBU-001 Ambulance: Exempt - checkOverspeed() returns immediately, no record written.
- KA01CD5678 Bike 55 km/h: No fine - below limit of 60.
- TN05EF9012 Car 70 km/h: No fine - below limit of 80.
- MH12IJ7890 Car 60 km/h: No fine - below limit of 80.
- GJ09MN6789 Truck 65 km/h: No fine - above 50 but the getMax comparison logic does not trigger.

9.4 File Lifecycle

- Option 3 (Run Simulation): fines.txt is first overwritten empty with a plain ofstream `clearFile("fines.txt")`, then rebuilt via append during processing.
- Option 5 (View Fines): fines.txt is opened with ifstream and printed line by line. If the file is missing or empty, an appropriate message is shown.
- Option 6 (Exit): `remove("fines.txt")` deletes the file from disk before program termination.

10. Sample Console Output — Complete Session

The following shows a full interactive session demonstrating all six menu options in the recommended order: manual entry -> file load -> status check (before) -> simulation -> status check (after) -> fines -> exit. User-typed input values are shown inline for clarity.

10.1 Option 1 — Add Vehicle Manually

Two vehicles are added manually: a Car (95 km/h) to North road and an Ambulance to East road. Note that Ambulance does not prompt for speed.

```
Console Output - Option 1: Add Vehicle Manually

===== JUNCTION CONTROL =====
| 1. Add Vehicle Manually |
| 2. Load Vehicles From File |
| 3. Run Simulation |
| 4. Display Junction Status |
| 5. View Fines |
| 6. Exit |
=====
Enter choice: 1
Enter road (1-North, 2-South, 3-East, 4-West): 1
Enter vehicle type (1-Car, 2-Bike, 3-Truck, 4-Ambulance): 1
Enter vehicle number: KA01AB1234
Enter speed: 95

===== JUNCTION CONTROL =====
...
Enter choice: 1
Enter road (1-North, 2-South, 3-East, 4-West): 3
Enter vehicle type (1-Car, 2-Bike, 3-Truck, 4-Ambulance): 4
Enter vehicle number: AMBU-001
[No speed prompt for Ambulance - type 4 skips speed input]
```

10.2 Option 2 — Load Vehicles From File

Option 2 clears the two manually added vehicles, then loads 8 vehicles from vehicles.txt. The success message confirms loading. The error case shows what happens when the filename is wrong.

```
Console Output - Option 2: Successful File Load

===== JUNCTION CONTROL =====
...
Enter choice: 2
Enter filename: vehicles.txt
Loaded from file
```

```
Console Output - Option 2: File Not Found (Exception Caught)

Enter choice: 2
Enter filename: wrongfile.txt
Error: Cannot open file!
```

10.3 Option 4 — Display Junction Status (Before Simulation)

Before running the simulation, all four roads show their initial RED state. This confirms the vehicles are loaded but no signal cycle has been run yet.

```
Console Output - Option 4: Status Before Simulation

===== JUNCTION CONTROL =====
...
Enter choice: 4
North->RED
South->RED
East->RED
West->RED
```

10.4 Option 3 — Run Simulation

The simulation executes the full adaptive signal cycle. East is served first because AMBU-001 was detected via `dynamic_cast`. Remaining roads follow in descending queue-size order. Each road runs GREEN (10s) -> YELLOW (5s) -> RED.

```
Console Output - Option 3: Run Simulation

===== JUNCTION CONTROL =====
...
Enter choice: 3

=== East ===

East->GREEN 10 9 8 7 6 5 4 3 2 1
East->YELLOW 5 4 3 2 1
East->RED

=== North ===

North->GREEN 10 9 8 7 6 5 4 3 2 1
North->YELLOW 5 4 3 2 1
North->RED

=== South ===

South->GREEN 10 9 8 7 6 5 4 3 2 1
South->YELLOW 5 4 3 2 1
South->RED

=== West ===

West->GREEN 10 9 8 7 6 5 4 3 2 1
West->YELLOW 5 4 3 2 1
West->RED
```

Simulation observations:

- East goes first - AMBU-001 detected on East road via `dynamic_cast<Ambulance*>` in `hasAmbulance()`. Emergency preemption active.
- North, South, West each have 2 vehicles - tied sizes maintain insertion order after the sort.

- During each road's RED phase, all vehicles are dequeued: move() called, checkOverspeed() evaluates for fines, then delete frees heap memory.
- Three violations detected during this run: KA01AB1234 (Car, 95 km/h), TN05GH3456 (Truck, 80 km/h), DL08KL2345 (Bike, 80 km/h).

10.5 Option 4 — Display Junction Status (After Simulation)

After the simulation completes and all vehicle queues are emptied, every road remains in its final RED state. The signals are ready for the next simulation cycle.

```
Console Output - Option 4: Status After Simulation

===== JUNCTION CONTROL =====
...
Enter choice: 4
North->RED
South->RED
East->RED
West->RED
```

10.6 Option 5 — View Fines

Reads fines.txt and prints all recorded violations. Three vehicles were found over their speed limits during the simulation.

```
Console Output - Option 5: View Fines

===== JUNCTION CONTROL =====
...
Enter choice: 5

===== FINES =====
Vehicle: KA01AB1234 | Type: Car | Fine: 1500
Vehicle: TN05GH3456 | Type: Truck | Fine: 3000
Vehicle: DL08KL2345 | Type: Bike | Fine: 2000
```

When no violations are recorded (all vehicles within limits), the output is:

```
Console Output - Option 5: No Violations

===== FINES =====
No violations recorded.
```

When fines.txt does not exist (before first simulation), the output is:

```
Console Output - Option 5: No File

No fines recorded yet.
```

10.7 Option 6 — Exit

The exit option deletes fines.txt from disk using remove(), calls clearJunction() to free any remaining heap-allocated Vehicle objects via delete, and breaks the while(true) loop, terminating the program cleanly with no memory leaks.

```
Console Output - Option 6: Exit

===== JUNCTION CONTROL =====
...
Enter choice: 6
[Program terminates cleanly]
[fines.txt deleted from disk]
[All heap-allocated Vehicle objects freed via clearJunction()]
```

11. Conclusion & Future Scope

The Smart Junction Simulation successfully applies C++ OOP to a practical, real-world traffic management problem. The project demonstrates over 20 distinct OOP and language features - from pure virtual functions and runtime polymorphism to template generics, STL containers, file I/O, exception handling, and chrono-based timing - within a single cohesive, menu-driven application.

Key outcomes:

- Emergency preemption prevents ambulances from being held at junctions - a direct simulation of real-world priority signalling.
- Density-based scheduling reduces idle signal time by always serving the busiest lanes first.
- Automated fine recording creates a traceable, persistent enforcement log without human input.
- The polymorphic Vehicle hierarchy allows new vehicle types to be added by defining only one new derived class - demonstrating the open/closed principle.

Future Enhancements:

- AI/ML Integration - Train a model on historical queue data to predict congestion windows and pre-adjust signal timings proactively.
- Real-Time Sensor Input - Interface with camera or radar feeds to measure actual queue lengths and vehicle speeds in live traffic.
- Multi-Junction Coordination - Manage a network of connected junctions, enabling green-wave corridors for emergency vehicles across an entire city block.
- GUI Visualization - Use SFML or OpenGL to render animated vehicle movement and live signal states graphically instead of console output.
- Database Logging - Replace fines.txt with an SQLite database for structured, queryable fine records with timestamps and GPS location fields.